💻 Programming

✅ Definition

Programming is the process of **writing instructions (code) that a computer can understand and execute to perform specific tasks**.

🔹 What is a Program?

✅ Definition

A **program** is a **set of instructions written in a programming language to solve a problem**.

👉 Example:

```
print("Hello World")
```

🔹 What is a Programming Language?

✅ Definition

A **programming language** is a **medium used to communicate with computers**.

👉 Examples:

- Python
- Java
- C++

🔹 Why Programming is Used?

- To **solve problems**
- To **automate tasks**
- To **build applications** (web, mobile, software)
- To **analyze data**

🔹 Basic Structure of a Program (Python)

```
# Input
name = input("Enter your name: ")

# Process
message = "Hello " + name

# Output
print(message)
```

🔹 Types of Programming Languages

1. High-Level Language

- Easy to understand
- Example: Python

2. Low-Level Language

- Close to machine language
- Hard to understand

🔹 Key Concepts in Programming

- Variables
- Data Types
- Operators
- Conditional Statements
- Loops
- Functions

📝 Interview Notes

- Programming = **writing instructions for computer**
- Program = **set of instructions**
- Python is a **high-level language**
- Used in **AI, Web, Data Science, Automation**
- Requires **logic building**

⚠️ Interview Trick Question

❓ Is programming and coding the same?

👉 **Answer:**

- **Coding** = writing code
- **Programming** = coding + logic + problem solving

🎯 Common Interview Questions

1. What is programming?
2. What is a program?
3. Difference between programming and coding?
4. What is a programming language?
5. Why is programming important?

🧠 One-Line Revision

👉 Programming = **giving instructions to computer to perform tasks**

⚙️ Compiler

✅ Definition

A **compiler** is a program that **translates the entire source code into machine code at once before execution**.

🔹 How it Works

1. Takes full program
2. Converts into machine code
3. Then executes

🔹 Example Languages

- C
- C++

✅ Advantages

- Faster execution
- Errors shown after full compilation

❌ Disadvantages

- Hard to debug (errors shown together)

🧠 Interpreter

✅ Definition

An **interpreter** is a program that **translates and executes code line by line**.

🔹 How it Works

1. Reads one line
2. Executes it
3. Moves to next line

🔹 Example Languages

- Python
- JavaScript

✅ Advantages

- Easy debugging
- Immediate execution

❌ Disadvantages

- Slower than compiler

⚖️ Difference Between Compiler & Interpreter

Feature	Compiler	Interpreter
Execution	Whole program	Line by line
Speed	Fast	Slower
Error Handling	After compilation	Immediate
Output	Machine code file	No separate file

⚠️ Interview Trick

❓ Is Python compiled or interpreted?

👉 **Answer:** Python is **both** ✅

- First compiled to **bytecode**
- Then interpreted by Python Virtual Machine (PVM)

🎯 Common Interview Questions

1. What is programming?
2. What is a compiler?
3. What is an interpreter?
4. Difference between compiler and interpreter?
5. Is Python compiled or interpreted?

🧠 One-Line Revision

- Programming → Writing instructions
- Compiler → Whole code → machine code
- Interpreter → Line by line execution

🐍 What is Python?

✅ Definition

Python is a **high-level, interpreted, general-purpose programming language** that is easy to learn and widely used for developing applications.

🔹 Simple Explanation

👉 Python is a language used to **communicate with computers and write programs** in a simple and readable way.

🔹 Example Program

```
print("Hello World")
```

👉 Output:

```
Hello World
```

🔹 Key Features of Python

✅ 1. Easy to Learn

- Simple syntax (like English)

✅ 2. Interpreted Language

- Executes code **line by line**

✅ 3. High-Level Language

- No need to manage memory manually

✅ 4. Portable

- Runs on **Windows, Mac, Linux**

✅ 5. Open Source

- Free to use

🔹 Where Python is Used

- 🌐 Web Development
- 📊 Data Science
- 🤖 Artificial Intelligence
- 🎮 Game Development
- 🔧 Automation / Scripting

📝 Interview Notes

- Python is **interpreted + high-level**
- Known for **readability**
- Uses **indentation instead of braces**
- Dynamically typed (no need to declare type)

⚠️ Interview Trick

❓ Why is Python popular?

👉 **Answer:**

- Easy syntax

- Large community
- Powerful libraries

🎯 Common Interview Questions

1. What is Python?
2. Is Python compiled or interpreted?
3. What are features of Python?
4. Why Python is easy to learn?
5. Where is Python used?

🧠 One-Line Revision

👉 Python = **easy, powerful, high-level programming language**

⚙️ Python Installation

✅ Definition

Python installation is the process of **downloading and setting up Python on your computer** so you can write and run Python programs.

🔹 Steps to Install Python (Windows)

👉 Step 1: Download Python

- Go to 👉 Python official website
- Click on **Download Python**

👉 Step 2: Run Installer

- Open the downloaded file

⚠️ **Important:** ✔️ Tick the option: **"Add Python to PATH"**

👉 Step 3: Install

- Click **Install Now**
- Wait for installation to complete

👉 Step 4: Verify Installation

Open Command Prompt and type:

```
python --version
```

👉 If installed correctly, it shows Python version.

🔹 Install on Mac / Linux

- Python usually comes **pre-installed**
- Check using:

```
python3 --version
```

🔹 How to Run Python

👉 1. Using Command Prompt

```
python
```

👉 2. Using IDLE (comes with Python)

- Open **IDLE**
- Write code and run

🔹 First Python Program

```
print("Hello World")
```

📝 Interview Notes

- Python must be **installed before coding**
- Always select **"Add to PATH"**
- `python --version` → check installation
- IDLE is **default Python editor**

⚠️ Interview Trick

❓ What happens if you don't add Python to PATH?

👉 **Answer:** You won't be able to run Python from command prompt directly.

🎯 Common Interview Questions

1. How do you install Python?
2. What is PATH in Python installation?
3. How to check Python version?
4. What is IDLE?
5. Can Python run without installation?

🧠 One-Line Revision

👉 Python installation = **setup Python to run programs on your system**

⚙️ Python Installation

✅ Definition

Python installation is the process of **downloading and setting up Python on your computer** so you can write and run Python programs.

🔹 Steps to Install Python (Windows)

👉 Step 1: Download Python

- Go to 👉 Python official website
- Click on **Download Python**

👉 Step 2: Run Installer

- Open the downloaded file

⚠️ **Important:** ✔️ Tick the option: **"Add Python to PATH"**

👉 Step 3: Install

- Click **Install Now**
- Wait for installation to complete

👉 Step 4: Verify Installation

Open Command Prompt and type:

```
python --version
```

👉 If installed correctly, it shows Python version.

🔹 Install on Mac / Linux

- Python usually comes **pre-installed**
- Check using:

```
python3 --version
```

🔹 How to Run Python

👉 1. Using Command Prompt

```
python
```

👉 2. Using IDLE (comes with Python)

- Open **IDLE**
- Write code and run

🔹 First Python Program

```
print("Hello World")
```

📝 Interview Notes

- Python must be **installed before coding**
- Always select **"Add to PATH"**
- `python --version` → check installation

- IDLE is **default Python editor**

⚠️ Interview Trick

❓ What happens if you don't add Python to PATH?

👉 **Answer:** You won't be able to run Python from command prompt directly.

🎯 Common Interview Questions

1. How do you install Python?
2. What is PATH in Python installation?
3. How to check Python version?
4. What is IDLE?
5. Can Python run without installation?

🧠 One-Line Revision

👉 Python installation = **setup Python to run programs on your system**

📦 Variables in Python

✅ Definition

A **variable** is a **name used to store data (value) in memory**.

🔹 Example

```
x = 10
name = "Python"
```

👉 `x` and `name` are variables

🔹 Rules of Identifiers (Variable Names)

✅ Definition

Identifiers are the names given to variables, functions, etc.

✔️ Valid Rules

1. Must start with a **letter (a-z, A-Z) or underscore (_)**
2. Cannot start with a number
3. Can contain **letters, digits, underscore**
4. Cannot use **keywords**
5. Case-sensitive (`name` ≠ `Name`)

👉 Examples

✔️ Valid:

```
my_name = "John"
age1 = 20
_total = 100
```

❌ Invalid:

```
1name = "John"     # starts with number
my-name = "John"   # special character
class = "Python"   # keyword
```

📊 Data Types in Python

✅ Definition

Data types define the **type of value a variable can store**.

🔹 Common Data Types

Data Type	Description	Example
`int`	Integer numbers	`10`
`float`	Decimal numbers	`10.5`
`str`	String (text)	`"Hello"`
`bool`	True/False	`True`
`list`	Collection (ordered)	`[1, 2, 3]`
`tuple`	Immutable list	`(1, 2, 3)`
`set`	Unordered unique values	`{1, 2, 3}`
`dict`	Key-value pairs	`{"a":1}`

🔹 Example

```
x = 10       # int
y = 10.5     # float
name = "Ram"  # string
```

📝 Interview Notes

- Python is **dynamically typed**
- No need to declare data type
- Type is decided at runtime

🔑 Keywords in Python

✅ Definition

Keywords are **reserved words in Python that have special meaning** and cannot be used as variable names.

🔹 Examples of Keywords

```
if, else, elif, for, while, break, continue,
def, return, True, False, None, class
```

🔹 Example

```
if x > 10:
    print("Greater")
```

👉 `if` is a keyword

⚠️ Important Rule

❌ You cannot use keywords as variable names:

```
class = "Python"   # ❌ Error
```

📝 Interview Notes

- Keywords are **reserved words**
- Used to define **syntax and structure**
- Cannot be used as identifiers

⚠️ Trick Questions

❓ Is `True` a keyword? 👉 Yes ✅

❓ Is `print` a keyword? 👉 No ❌ (it is a function)

🎯 Common Interview Questions

1. What is a variable?
2. What are identifier rules?
3. What are data types?
4. What are keywords?
5. Can we use keywords as variable names?
6. Is Python statically or dynamically typed?

🧠 One-Line Revision

- Variable → stores data
- Identifier → name of variable
- Data type → type of value
- Keyword → reserved word

💬 Comments in Python

✅ Definition

Comments are **non-executable lines used to explain code**.

🔹 Types of Comments

👉 1. Single-line Comment

```
# This is a comment
print("Hello")
```

👉 2. Multi-line Comment

```
"""
This is
multi-line comment
"""
```

📝 Interview Notes

- Comments are ignored by Python
- Used for **readability & explanation**
- Do not affect program execution

➕ Types of Operators in Python

✅ Definition

Operators are symbols used to **perform operations on variables and values**.

🔹 1. Arithmetic Operators

```
+ - * / % // **
```

👉 Example:

```
a = 10
b = 3
print(a + b)   # 13
```

🔹 2. Comparison Operators

```
== != > < >= <=
```

🔹 3. Logical Operators

```
and or not
```

🔹 4. Assignment Operators

```
= += -= *= /=
```

🔹 5. Membership Operators

```
in   not in
```

🔹 6. Identity Operators

```
is   is not
```

📝 Interview Notes

- Operators perform **calculations & comparisons**
- `//` → floor division
- `**` → power

🔄 Type Conversion & Type Casting

✅ Definition

Type conversion means **changing one data type into another**.

🔹 1. Type Conversion (Implicit)

✅ Definition

Automatic conversion by Python.

```
x = 5
y = 2.5
print(x + y)   # 7.5 (int → float)
```

🔹 2. Type Casting (Explicit)

✅ Definition

Manual conversion using functions.

🔹 Common Casting Functions

```
int()
float()
str()
```

👉 Example

```
x = "10"
y = int(x)

print(y + 5)   # 15
```

📝 Interview Notes

- Implicit = automatic
- Explicit = manual
- Used to avoid **type errors**

⌨️ Input in Python

✅ Definition

Input is used to **take data from the user during program execution**.

🔹 Using `input()`

```
name = input("Enter your name: ")
print(name)
```

🔹 Input with Type Casting

```
age = int(input("Enter age: "))
print(age + 5)
```

⚠️ Important Point

👉 `input()` always takes **string by default**

📝 Interview Notes

- `input()` → user input

- Always returns **string**
- Use `int()` / `float()` for numbers

⚠️ Trick Questions

❓ Output?

```
x = input("Enter: ")
print(type(x))
```

👉 **Answer:** `<class 'str'>`

❓ Output?

```
print(10 / 3)
```

👉 **Answer:** `3.333...`

🎯 Common Interview Questions

1. What are comments?
2. Types of operators?
3. Difference between type casting & conversion?
4. What does `input()` return?
5. Why do we use type casting?

🧠 One-Line Revision

- Comments → explanation in code
- Operators → perform operations
- Type conversion → change data type
- Input → take user data

🧵 Strings in Python

✅ Definition

A **string** in Python is a **sequence of characters enclosed within quotes** (single, double, or triple quotes).

👉 Example:

```
name = "Python"
```

🔹 1. Basic Operations on Strings

✅ Definition

Basic operations are the **common actions performed on strings**, such as accessing characters, slicing, and checking values.

👉 Indexing (Access characters)

```
text = "Python"
print(text[0])   # P
```

👉 Slicing (Get part of string)

```
print(text[1:4])   # yth
```

👉 Negative Indexing

```
print(text[-1])   # n
```

🔹 2. Concatenation

✅ Definition

Concatenation means **joining two or more strings into one string**.

👉 Example:

```
a = "Hello"
b = "World"

print(a + " " + b)   # Hello World
```

🔹 3. Length of String

✅ Definition

The **length of a string** is the **total number of characters present in the string**, including spaces.

👉 Example:

```
text = "Python"
print(len(text))   # 6
```

📝 Interview Notes (Very Important)

- String = **sequence of characters**
- Strings are **immutable** ❗
- Indexing starts from **0**
- Supports **positive & negative indexing**
- `+` → used for concatenation
- `len()` → used to find length

⚠️ Interview Trick Question

❓ What will happen?

```
s = "hello"
s[0] = "H"
```

👉 **Answer:** ❌ Error (because strings are immutable)

🎯 Short One-Line Revision

- String → Collection of characters
- Concatenation → Joining strings
- Length → Number of characters using `len()`

🔤 Escape Sequence Characters in Python

✅ Definition

An **escape sequence character** is a **special character preceded by a backslash (`\`)** used to represent characters that are difficult or impossible to type directly.

🔹 Common Escape Sequences

Escape Sequence	Meaning
`\n`	New line
`\t`	Tab (space)
`\\`	Backslash
`\'`	Single quote
`\"`	Double quote

🔹 Examples

👉 1. New Line (`\n`)

```
print("Hello\nWorld")
```

👉 Output:

```
Hello
World
```

👉 2. Tab Space (`\t`)

```
print("Hello\tWorld")
```

👉 Output:

```
Hello	World
```

👉 3. Using Quotes Inside String

```
print("He said \"Hello\"")
```

👉 Output:

```
He said "Hello"
```

👉 4. Backslash (`\\`)

```
print("C:\\Users\\Admin")
```

👉 Output:

```
C:\Users\Admin
```

📝 Interview Notes (Important)

- Escape sequences start with `\` **(backslash)**
- Used for **formatting strings**
- Helps include **special characters inside strings**
- Very useful in **file paths and printing formatted output**

⚠️ Interview Trick

❓ What will be the output?

```
print("Hello\World")
```

👉 **Answer:** May give unexpected output or warning 👉 Correct way:

```
print("Hello\\World")
```

🎯 Common Interview Questions

1. What is an escape sequence?
2. What does `\n` do?
3. Difference between `\n` and `\t` ?
4. How to print quotes inside a string?
5. Why use `\\` in file paths?

🧠 One-Line Revision

👉 Escape sequence = **special character using `\` for formatting or representation**

🔢 Indexing in Python (Strings)

✅ Definition

Indexing is the process of **accessing individual characters of a string using their position (index number)**.

🔹 How Indexing Works

In Python, indexing **starts from 0**.

```
text = "Python"
```

Character	P	y	t	h	o	n
Index	0	1	2	3	4	5

🔹 1. Positive Indexing

✅ Definition

Accessing characters from **left to right starting from 0**.

👉 Example:

```
text = "Python"
print(text[0])   # P
print(text[3])   # h
```

🔹 2. Negative Indexing

✅ Definition

Accessing characters from **right to left starting from -1**.

Character	P	y	t	h	o	n
Index	-6	-5	-4	-3	-2	-1

👉 Example:

```
text = "Python"
print(text[-1])   # n
print(text[-3])   # h
```

🔹 3. Indexing with Variables

```
text = "Python"
i = 2
print(text[i])   # t
```

⚠️ Important Rule (Interview)

❗ If index is out of range → Error

```
text = "Python"
print(text[10])   # ❌ IndexError
```

📝 Interview Notes

- Index starts from **0**
- Negative indexing starts from **-1**
- Used to **access single character**
- Strings are **immutable** (cannot change using index)
- Out-of-range index → **IndexError**

⚠️ Trick Question

❓ What will be the output?

```
text = "Hello"
print(text[-0])
```

👉 **Answer:** `H` 👉 Because `-0 = 0`

🎯 Common Interview Questions

1. What is indexing in Python?
2. Difference between **positive and negative indexing**?
3. What happens if index is out of range?
4. Can we modify a string using indexing?

5. Why does indexing start from 0?

🧠 One-Line Revision

👉 Indexing = **access characters using position (0, 1, 2, …)**

✂️ Slicing in Python (Strings)

✅ Definition

Slicing is the process of **extracting a part (substring) of a string using a range of indexes**.

🔹 Syntax

```
string[start : end : step]
```

- **start** → starting index (included)
- **end** → ending index (excluded)
- **step** → jump (optional)

🔹 1. Basic Slicing

```
text = "Python"
print(text[0:4])   # Pyth
```

👉 Takes characters from index **0 to 3**

🔹 2. Omitting Start or End

```
text = "Python"

print(text[:3])   # Pyt  (start = 0)
print(text[2:])   # thon (till end)
```

🔹 3. Negative Index Slicing

```
text = "Python"

print(text[-4:-1])   # tho
```

🔹 4. Step in Slicing

```
text = "Python"

print(text[0:6:2])   # Pto
```

👉 Skips characters (step = 2)

🔹 5. Reverse String

```
text = "Python"

print(text[::-1])   # nohtyP
```

⚠️ Important Rules (Interview)

- Start index is **included**
- End index is **excluded**
- Default values:
  - start = 0
  - end = length of string
- No error if range is out → returns empty string

```
print("Python"[10:20])   # ""
```

📝 Interview Notes

- Slicing = **substring extraction**
- Syntax → `[start:end:step]`
- Works with **positive & negative indexes**
- Step can be used to **skip or reverse**
- Does NOT modify original string (immutable)

⚠️ Trick Questions

❓ Output?

```
text = "Python"
print(text[1:5:2])
```

👉 **Answer:** `yh`

❓ Output?

```
print("Python"[::-1])
```

👉 **Answer:** `nohtyP`

🎯 Common Interview Questions

1. What is slicing in Python?
2. Difference between indexing and slicing?
3. What does `text[::-1]` do?
4. Why is end index excluded?
5. What happens if slice is out of range?

🧠 One-Line Revision

👉 Slicing = **getting part of string using `[start:end:step]`**

🔧 String Functions in Python

✅ Definition

String functions (methods) are **built-in functions used to perform operations on strings**, such as changing case, searching, replacing, etc.

🔹 Common String Functions

👉 1. `lower()`

✅ Definition

Converts all characters to **lowercase**.

```
text = "HELLO"
print(text.lower())   # hello
```

👉 2. `upper()`

✅ Definition

Converts all characters to **uppercase**.

```
text = "hello"
print(text.upper())   # HELLO
```

👉 3. `strip()`

✅ Definition

Removes **spaces from beginning and end**.

```
text = "  hello  "
print(text.strip())   # hello
```

👉 4. `replace()`

✅ Definition

Replaces a **word or character with another**.

```
text = "Hello World"
print(text.replace("World", "Python"))
# Hello Python
```

👉 5. `find()`

✅ Definition

Returns the **index of first occurrence** of a substring.

```
text = "Python"
print(text.find("t"))   # 2
```

👉 If not found → returns `-1`

👉 6. `count()`

✅ Definition

Counts how many times a character/word appears.

```
text = "banana"
print(text.count("a"))   # 3
```

👉 7. `startswith()`

✅ Definition

Checks if string starts with given value.

```
text = "Python"
print(text.startswith("Py"))   # True
```

👉 8. `endswith()`

✅ Definition

Checks if string ends with given value.

```
text = "Python"
print(text.endswith("on"))   # True
```

👉 9. `split()`

✅ Definition

Splits string into a **list**.

```
text = "Hello World"
print(text.split())   # ['Hello', 'World']
```

👉 10. `join()`

✅ Definition

Joins elements of a list into a string.

```
words = ["Hello", "World"]
print(" ".join(words))   # Hello World
```

📝 Interview Notes

- String functions are **built-in methods**
- They **do not modify original string** (immutable)
- Always return a **new string**
- `find()` returns **-1 if not found**
- `split()` → string → list
- `join()` → list → string

⚠️ Trick Questions

❓ Output?

```
text = "Hello"
print(text.replace("l", "L"))
```

👉 **Answer:** `HeLLo`

❓ Output?

```
print("abc".find("z"))
```

👉 **Answer:** `-1`

🎯 Common Interview Questions

1. What are string functions?
2. Difference between `find()` and `index()` ?
3. What does `split()` do?
4. Explain `join()` with example.
5. Why strings are immutable but functions still work?

🧠 One-Line Revision

👉 String functions = **built-in methods to manipulate strings**

🔧 Other String Functions in Python

👉 1. `capitalize()`

✅ Definition

Converts **first character to uppercase**.

```
text = "hello world"
print(text.capitalize())   # Hello world
```

👉 2. `title()`

✅ Definition

Converts **first letter of each word to uppercase**.

```
text = "hello world"
print(text.title())   # Hello World
```

👉 3. `swapcase()`

✅ Definition

Converts **uppercase to lowercase and vice versa**.

```
text = "Hello"
print(text.swapcase())   # hELLO
```

👉 4. `isalpha()`

✅ Definition

Returns **True if all characters are alphabets**.

```
print("Hello".isalpha())   # True
print("Hello123".isalpha()) # False
```

👉 5. `isdigit()`

✅ Definition

Returns **True if all characters are digits**.

```
print("123".isdigit())   # True
print("123a".isdigit())  # False
```

👉 6. `isalnum()`

✅ Definition

Returns **True if string is alphanumeric (letters + numbers)**.

```
print("abc123".isalnum())   # True
```

👉 7. `isspace()`

✅ Definition

Returns **True if string contains only spaces**.

```
print("   ".isspace())   # True
```

👉 8. `center()`

✅ Definition

Aligns string to center with padding.

```
text = "Hi"
print(text.center(10, "-"))
# ----Hi----
```

👉 9. `ljust()` and `rjust()`

✅ Definition

Align string to left or right.

```
text = "Hi"
print(text.ljust(5, "-"))   # Hi---
print(text.rjust(5, "-"))   # ---Hi
```

👉 10. `zfill()`

✅ Definition

Adds zeros at beginning.

```
print("5".zfill(3))   # 005
```

📝 Interview Notes

- `isalpha()` , `isdigit()` , `isalnum()` → **validation functions**
- `capitalize()` , `title()` → **formatting**
- `center()` , `ljust()` , `rjust()` → **alignment**
- `zfill()` → useful in **numbers & IDs**
- All functions return **new string (immutability)**

⚠️ Trick Questions

❓ Output?

```
print("hello world".title())
```

👉 **Answer:** `Hello World`

❓ Output?

```
print("123".isalpha())
```

👉 **Answer:** `False`

🎯 Common Interview Questions

1. Difference between `isalpha()` and `isalnum()` ?
2. What does `isdigit()` check?
3. Difference between `capitalize()` and `title()` ?
4. What is `zfill()` used for?
5. Which functions are used for validation?

🧠 One-Line Revision

👉 Extra string functions = **used for validation, formatting, and alignment**

🔀 Conditional Statements in Python

✅ Definition

Conditional statements are used to **make decisions in a program based on conditions**.

👉 They execute different blocks of code depending on whether a condition is **True or False**.

🔹 Types of Conditional Statements

1. `if`
2. `if-else`
3. `if-elif-else`
4. Nested `if`

🔹 1. `if` Statement

✅ Definition

Executes code **only if condition is True**.

```
age = 18

if age >= 18:
    print("You can vote")
```

🔹 2. `if-else` Statement

✅ Definition

Executes one block if condition is True, otherwise another block.

```
age = 16

if age >= 18:
    print("Eligible")
else:
    print("Not Eligible")
```

🔹 3. `if-elif-else` Statement

✅ Definition

Used when there are **multiple conditions**.

```
marks = 75

if marks >= 90:
    print("A Grade")
elif marks >= 70:
    print("B Grade")
else:
    print("C Grade")
```

🔹 4. Nested `if`

✅ Definition

An `if` statement inside another `if`.

```
age = 20
citizen = True

if age >= 18:
    if citizen:
        print("Eligible to vote")
```

🔹 Comparison Operators (Used in Conditions)

Operator	Meaning
`==`	Equal to
`!=`	Not equal
`>`	Greater than
`<`	Less than
`>=`	Greater or equal
`<=`	Less or equal

🔹 Logical Operators

Operator	Meaning
`and`	Both conditions True
`or`	At least one True
`not`	Opposite

```
age = 20
if age > 18 and age < 60:
    print("Working age")
```

📝 Interview Notes

- Used for **decision making**
- Condition must return **True/False**
- Indentation is **very important in Python**
- `elif` = else if
- You can use **multiple conditions**

⚠️ Trick Questions

❓ Output?

```
if 0:
    print("Hello")
else:
    print("World")
```

👉 **Answer:** `World` 👉 Because `0` is treated as **False**

❓ Output?

```
if "":
    print("Hi")
else:
    print("Bye")
```

👉 **Answer:** `Bye` 👉 Empty string = **False**

🎯 Common Interview Questions

1. What are conditional statements?
2. Difference between `if` and `if-else`?
3. What is `elif`?
4. Why indentation is important in Python?
5. Can we use multiple conditions in `if`?

🧠 One-Line Revision

👉 Conditional statements = **used to make decisions based on conditions**

🎨 String Formatting in Python

✅ Definition

String formatting is the process of **inserting variables or values into a string in a structured and readable way**.

🔹 Types of String Formatting

1. **Using `%` (Old style)**
2. **Using `format()`**
3. **Using f-strings (Modern & Best)**

🔹 1. `%` Formatting (Old Method)

✅ Definition

Uses `%` operator to insert values into string.

```
name = "John"
age = 25

print("My name is %s and I am %d years old" % (name, age))
```

👉 Output:

```
My name is John and I am 25 years old
```

🔹 2. `format()` Method

✅ Definition

Uses `{}` placeholders and `.format()` function.

```
name = "John"
age = 25

print("My name is {} and I am {} years old".format(name, age))
```

👉 With Index

```
print("Name: {0}, Age: {1}".format("John", 25))
```

🔹 3. f-Strings (Recommended ✅)

✅ Definition

Uses `f` before string and `{}` to directly insert variables.

```
name = "John"
age = 25

print(f"My name is {name} and I am {age} years old")
```

🔹 Expressions in f-strings

```
a = 10
b = 5

print(f"Sum = {a + b}")   # Sum = 15
```

📝 Interview Notes

- `%` → old method
- `format()` → flexible
- f-string → **fastest & easiest (Python 3.6+)**
- Use `{}` for placeholders

⚠️ Trick Questions

❓ Output?

```
name = "Ram"
print(f"Hello {name}")
```

👉 **Answer:** `Hello Ram`

❓ Output?

```
print("Value: {}".format(10 + 5))
```

👉 **Answer:** `Value: 15`

🎯 Common Interview Questions

1. What is string formatting?
2. Difference between `%`, `format()`, and f-string?
3. Which is best method?
4. Can we use expressions in f-strings?
5. What are placeholders?

🧠 One-Line Revision

👉 String formatting = **inserting values into string using `%`, `format()`, or f-string**

🎨 Expressions in f-Strings with Formatting Types 👇

✅ Definition

Expressions in f-strings with formatting types means using **expressions inside `{}` along with format specifiers** to control how the output looks (like decimal places, alignment, etc.).

🔹 Basic Syntax

```
f"{expression:format_specifier}"
```

🔹 1. Integer Formatting

👉 Example

```
num = 10
print(f"Number: {num:d}")
```

👉 Output:

```
Number: 10
```

🔹 2. Float Formatting (Decimal Places)

👉 Example

```
pi = 3.14159
print(f"Value: {pi:.2f}")
```

👉 Output:

```
Value: 3.14
```

👉 `.2f` → 2 decimal places

🔹 3. Width Formatting

```
num = 5
print(f"{num:5}")
```

👉 Output:

```
    5
```

👉 Adds spaces (right aligned)

🔹 4. Left / Right Alignment

```
text = "Hi"

print(f"{text:<10}")   # Left
print(f"{text:>10}")   # Right
```

🔹 5. Center Alignment

```
print(f"{text:^10}")
```

🔹 6. Fill Characters

```
print(f"{text:-^10}")
```

👉 Output:

```
----Hi----
```

🔹 7. Expression + Formatting

```
a = 10
b = 3

print(f"Result: {(a/b):.2f}")
```

👉 Output:

```
Result: 3.33
```

🔹 8. Percentage Formatting

```
score = 0.85
print(f"Score: {score:.0%}")
```

👉 Output:

```
Score: 85%
```

🔹 9. Number with Commas

```
num = 1000000
print(f"{num:,}")
```

👉 Output:

```
1,000,000
```

🔹 10. Binary, Octal, Hex

```
num = 10

print(f"Binary: {num:b}")
print(f"Octal: {num:o}")
print(f"Hex: {num:x}")
```

📝 Interview Notes

- f-strings support **expressions + formatting**
- Syntax → `{value:format}`

- `.2f` → decimal control
- `< > ^` → alignment
- `,` → comma separator
- `%` → percentage

⚠️ Trick Questions

❓ Output?

```
x = 3.4567
print(f"{x:.1f}")
```

👉 **Answer:** `3.5`

❓ Output?

```
print(f"{10:04}")
```

👉 **Answer:** `0010`

🎯 Common Interview Questions

1. What is f-string formatting?
2. What does `.2f` mean?
3. How to align text in f-strings?
4. How to format percentage?
5. How to format large numbers?

🧠 One-Line Revision

👉 f-string formatting = **control output using** `{expression:format}`

📋 List in Python

✅ Definition

A **list** is a **collection of multiple items stored in a single variable**, enclosed in square brackets `[]`.

🔹 Example

```
numbers = [10, 20, 30, 40, 50]
```

🔪 Slicing in List

✅ Definition

Slicing is used to **extract a part of a list using index range**.

🔹 Syntax

```
list[start : end : step]
```

🔹 Positive Slicing

✅ Definition

Accessing elements **from left to right (starting from index 0)**.

👉 Example

```
nums = [10, 20, 30, 40, 50]

print(nums[1:4])   # [20, 30, 40]
```

👉 Index 1 to 3 (end excluded)

👉 More Examples

```
print(nums[:3])   # [10, 20, 30]
print(nums[2:])   # [30, 40, 50]
```

🔹 Negative Slicing

✅ Definition

Accessing elements **from right to left (starting from -1)**.

👉 Index View

Element	10	20	30	40	50
Index	0	1	2	3	4
Neg	-5	-4	-3	-2	-1

👉 Example

```
nums = [10, 20, 30, 40, 50]

print(nums[-4:-1])   # [20, 30, 40]
```

🔹 Step in Slicing

```
print(nums[0:5:2])   # [10, 30, 50]
```

🔹 Reverse List

```
print(nums[::-1])   # [50, 40, 30, 20, 10]
```

⚠️ Important Rules (Interview)

- Start index → **included**
- End index → **excluded**
- Works with **positive & negative indexes**
- No error if out of range → returns empty list

```
print(nums[10:20])   # []
```

📝 Interview Notes

- List slicing = **extract part of list**
- Syntax → `[start:end:step]`
- Negative indexing starts from **-1**
- Step used to **skip or reverse**
- Does NOT change original list

⚠️ Trick Questions

❓ Output?

```
nums = [1, 2, 3, 4, 5]
print(nums[1:5:2])
```

👉 **Answer:** `[2, 4]`

❓ Output?

```
print([1,2,3][::-1])
```

👉 **Answer:** `[3, 2, 1]`

🎯 Common Interview Questions

1. What is list slicing?
2. Difference between positive and negative slicing?
3. What does `[::-1]` do?
4. Why is end index excluded?
5. What happens if index is out of range?

🧠 One-Line Revision

👉 List slicing = **getting part of list using `[start:end:step]`**

Here's a **simple + interview-ready explanation** of **List Methods in Python** 👇

📋 List Methods in Python

✅ Definition

List methods are **built-in functions used to perform operations on lists**, such as adding, removing, sorting, etc.

🔹 Common List Methods

👉 1. `append()`

✅ Definition

Adds an element **at the end** of the list.

```
nums = [1, 2, 3]
nums.append(4)
print(nums)   # [1, 2, 3, 4]
```

👉 2. `insert()`

✅ Definition

Adds an element at a **specific position**.

```
nums = [1, 2, 3]
nums.insert(1, 10)
print(nums)   # [1, 10, 2, 3]
```

👉 3. `remove()`

✅ Definition

Removes a **specific value**.

```
nums = [1, 2, 3]
nums.remove(2)
print(nums)   # [1, 3]
```

👉 4. `pop()`

✅ Definition

Removes element using **index** (default last).

```
nums = [1, 2, 3]
nums.pop()
print(nums)   # [1, 2]
```

👉 5. `clear()`

✅ Definition

Removes **all elements**.

```
nums = [1, 2, 3]
nums.clear()
print(nums)   # []
```

👉 6. `index()`

✅ Definition

Returns the **index of an element**.

```
nums = [10, 20, 30]
print(nums.index(20))   # 1
```

👉 7. `count()`

✅ Definition

Counts occurrences of an element.

```
nums = [1, 2, 2, 3]
print(nums.count(2))   # 2
```

👉 8. `sort()`

✅ Definition

Sorts the list.

```
nums = [3, 1, 2]
nums.sort()
print(nums)   # [1, 2, 3]
nums.sort(reverse = True)
print(nums)   # [3, 2, 1]
```

👉 9. `reverse()`

✅ Definition

Reverses the list.

```
nums = [1, 2, 3]
nums.reverse()
print(nums)   # [3, 2, 1]
```

👉 10. `copy()`

✅ Definition

Creates a **copy of the list**.

```
nums = [1, 2, 3]
new_list = nums.copy()
```

📝 Interview Notes

- Lists are **mutable** (can be changed)
- Methods like `append()`, `remove()` modify original list
- `sort()` changes original list (in-place)
- `pop()` returns removed value

⚠️ Trick Questions

❓ Output?

```
nums = [1, 2, 3]
print(nums.pop())
```

👉 **Answer:** `3`

❓ Output?

```
nums = [3, 1, 2]
print(nums.sort())
```

👉 **Answer:** `None` 👉 Because `sort()` modifies list but returns nothing

---

🎯 Common Interview Questions

1. What are list methods?
2. Difference between `append()` and `insert()`?
3. Difference between `remove()` and `pop()`?

4. What does `sort()` return?
5. Are lists mutable?

🧠 One-Line Revision

👉 List methods = **functions to modify and manage list data**

📦 Tuple in Python

✅ Definition

A **tuple** is a **collection of elements stored in a single variable**, enclosed in parentheses `()` and **immutable (cannot be changed)**.

🔹 Example

```
t = (10, 20, 30, 40, 50)
```

🔪 Slicing in Tuple

✅ Definition

Slicing is used to **extract a part of a tuple using index range**.

🔹 Syntax

```
tuple[start : end : step]
```

🔹 Positive Slicing

✅ Definition

Access elements **from left to right (index starts from 0)**.

👉 Example

```
t = (10, 20, 30, 40, 50)

print(t[1:4])   # (20, 30, 40)
```

👉 More Examples

```
print(t[:3])   # (10, 20, 30)
print(t[2:])   # (30, 40, 50)
```

🔹 Negative Slicing

✅ Definition

Access elements **from right to left (index starts from -1)**.

👉 Index View

Element	10	20	30	40	50
Index	0	1	2	3	4
Neg	-5	-4	-3	-2	-1

👉 Example

```
print(t[-4:-1])   # (20, 30, 40)
```

🔹 Step in Slicing

```
print(t[0:5:2])   # (10, 30, 50)
```

🔹 Reverse Tuple

```
print(t[::-1])   # (50, 40, 30, 20, 10)
```

⚠️ Important Rules (Interview)

- Start index → **included**
- End index → **excluded**

- Works with **positive & negative indexing**
- No error if out of range → returns empty tuple

```
print(t[10:20])   # ()
```

📝 Interview Notes

- Tuple slicing = **extract part of tuple**
- Syntax → `[start:end:step]`
- Tuple is **immutable** ❗
- Slicing creates **new tuple**

⚠️ Trick Questions

❓ Output?

```
t = (1, 2, 3, 4, 5)
print(t[1:5:2])
```

👉 **Answer:** `(2, 4)`

❓ Output?

```
print((1,2,3)[::-1])
```

👉 **Answer:** `(3, 2, 1)`

🎯 Common Interview Questions

1. What is a tuple?
2. Difference between list and tuple?
3. What is slicing in tuple?
4. Is tuple mutable?
5. What does `[::-1]` do?

🧠 One-Line Revision

👉 Tuple slicing = **getting part of tuple using `[start:end:step]`**

📦 Tuple with Single Value in Python

✅ Definition

A **single-value tuple** is a tuple that contains **only one element**, and it must have a **comma (,) after the value**.

🔹 Why Comma is Important?

👉 Without a comma, Python treats it as a **normal value**, not a tuple.

🔹 Example

✔️ Correct (Single-value tuple)

```
t = (10,)
print(type(t))   # <class 'tuple'>
```

❌ Wrong (Not a tuple)

```
t = (10)
print(type(t))   # <class 'int'>
```

🔹 More Examples

```
t1 = ("Hello",)
t2 = (True,)
t3 = (5.5,)
```

🔹 Accessing Value

```
t = (10,)
print(t[0])   # 10
```

⚠️ Important Points (Interview)

- Must use **comma (,)**
- Parentheses alone are **not enough**
- Tuple is still **immutable**
- Can store **any data type**

📝 Interview Notes

- `(value,)` → tuple
- `(value)` → normal value
- Very common **trick question** ❗

⚠️ Trick Question

❓ Output?

```
t = (5)
print(type(t))
```

👉 **Answer:** `<class 'int'>`

❓ Output?

```
t = (5,)
print(type(t))
```

👉 **Answer:** `<class 'tuple'>`

🎯 Common Interview Questions

1. How to create a single-value tuple?
2. Why is comma required in tuple?
3. Difference between `(5)` and `(5,)` ?
4. Is tuple mutable?

🧠 One-Line Revision

👉 Single-value tuple = **(value,) with comma**

📦 Tuple Methods in Python

✅ Definition

Tuple methods are **built-in functions used to perform operations on tuples**.

👉 Since tuples are **immutable**, they have **very few methods**.

🔹 Tuple Methods (Only 2 Important)

👉 1. `count()`

✅ Definition

Returns the **number of times an element appears in the tuple**.

```
t = (1, 2, 2, 3, 2)
print(t.count(2))   # 3
```

👉 2. `index()`

✅ Definition

Returns the **index of the first occurrence of an element**.

```
t = (10, 20, 30, 20)
print(t.index(20))   # 1
```

🔹 Additional Operations (Not Methods but Useful)

👉 1. `len()`

✅ Definition

Returns the **number of elements in tuple**.

```
t = (1, 2, 3)
print(len(t))   # 3
```

👉 2. `max()` and `min()`

```
t = (10, 5, 20)
print(max(t))   # 20
print(min(t))   # 5
```

👉 3. `sum()`

```
t = (1, 2, 3)
print(sum(t))   # 6
```

📝 Interview Notes

- Tuples are **immutable** ❗
- Only **2 methods** → `count()` **and** `index()`
- Other functions like `len()` , `sum()` are **built-in functions**, not tuple methods
- Faster than lists

⚠️ Trick Questions

❓ Output?

```
t = (1, 2, 3)
t.append(4)
```

👉 **Answer:** ❌ Error (tuples don't support `append()`)

❓ Output?

```
t = (1, 2, 2, 3)
print(t.index(2))
```

👉 **Answer:** `1` (first occurrence)

🎯 Common Interview Questions

1. What are tuple methods?
2. Why tuple has fewer methods than list?
3. Difference between `count()` and `index()` ?
4. Can we modify a tuple?
5. Is tuple faster than list?

🧠 One-Line Revision

👉 Tuple methods = **only** `count()` **and** `index()`

📘 Dictionary in Python (Complete Notes)

✅ Definition

A **dictionary** is a **collection of data stored in key-value pairs**, enclosed in `{}`.

👉 Each element has:

- **Key** → unique identifier
- **Value** → data

🔹 Example

```
student = {
    "name": "John",
    "age": 20,
    "marks": 85
}
```

🔹 Characteristics

- Stored as **key : value**
- Keys must be **unique**
- Values can be **any data type**
- Dictionary is **mutable**
- Fast access using keys

🔹 Basic Operations

👉 Access

```
print(student["name"])   # John
```

👉 Add / Update

```
student["age"] = 21
student["city"] = "Delhi"
```

👉 Delete

```
student.pop("age")
```

🔧 Dictionary Methods (with Description + Syntax)

👉 1. `get()`

✅ Description

Returns value safely (no error if key not found)

🔹 Syntax

```
dict.get(key, default)
```

👉 Example

```
print(student.get("name"))   # John
print(student.get("age", 0)) # 0
```

👉 2. `keys()`

✅ Description

Returns all keys

🔹 Syntax

```
dict.keys()
```

👉 3. `values()`

✅ Description

Returns all values

🔹 Syntax

```
dict.values()
```

👉 4. `items()`

✅ Description

Returns key-value pairs

🔹 Syntax

```
dict.items()
```

👉 5. `update()`

✅ Description

Adds or updates values

🔹 Syntax

```
dict.update({key: value})
```

👉 6. `pop()`

✅ Description

Removes element by key

🔹 Syntax

```
dict.pop(key)
```

👉 7. `popitem()`

✅ Description

Removes last inserted item

🔹 Syntax

```
dict.popitem()
```

👉 8. `clear()`

✅ Description

Removes all items

🔹 Syntax

```
dict.clear()
```

👉 9. `copy()`

✅ Description

Creates a copy of dictionary

🔹 Syntax

```
dict.copy()
```

👉 10. `setdefault()`

✅ Description

Returns value; if not present, adds key with default value

🔹 Syntax

```
dict.setdefault(key, default)
```

🔹 Looping in Dictionary

```
for key, value in student.items():
    print(key, value)
```

📝 Interview Notes

- Dictionary = **key-value pair**
- Keys must be **unique**
- Mutable (can change)

- `get()` is safer than `[]`
- `popitem()` removes last element
- `setdefault()` adds key if not present

⚠️ Trick Questions

❓ Output?

```
d = {"a": 1, "a": 2}
print(d)
```

👉 **Answer:** `{'a': 2}`

❓ Output?

```
d = {"x": 10}
print(d.get("y"))
```

👉 **Answer:** `None`

🎯 Common Interview Questions

1. What is a dictionary?
2. Why keys must be unique?
3. Difference between `get()` and `[]` ?
4. What does `update()` do?
5. What is `setdefault()` ?

🧠 One-Line Revision

👉 Dictionary = **mutable collection of key-value pairs** `{key: value}`

🟢 Set in Python (Complete Notes)

✅ Definition

A **set** is a **collection of unique elements**, enclosed in `{}` .

👉 It **does not allow duplicate values** and is **unordered**.

🔹 Example

```
s = {1, 2, 3, 4}
```

🔹 Characteristics

- No **duplicate values**
- **Unordered** (no indexing)
- Mutable (can add/remove items)
- Elements must be **immutable** (int, str, tuple)

🔹 Basic Operations

👉 Add Element

```
s.add(5)
```

👉 Remove Element

```
s.remove(2)
```

👉 Check Element

```
print(3 in s)   # True
```

🔧 Set Methods (Description + Syntax)

👉 1. `add()`

✅ Description

Adds an element to the set

🔹 Syntax

```
set.add(value)
```

👉 2. `remove()`

✅ Description

Removes element (error if not found)

🔹 Syntax

```
set.remove(value)
```

👉 3. `discard()`

✅ Description

Removes element (no error if not found)

🔹 Syntax

```
set.discard(value)
```

👉 4. `pop()`

✅ Description

Removes a random element

🔹 Syntax

```
set.pop()
```

👉 5. `clear()`

✅ Description

Removes all elements

🔹 Syntax

```
set.clear()
```

👉 6. `union()`

✅ Description

Combines two sets

🔹 Syntax

```
set1.union(set2)
```

👉 Example

```
a = {1, 2}
b = {2, 3}
print(a.union(b))   # {1, 2, 3}
```

👉 7. `intersection()`

✅ Description

Common elements in both sets

🔹 Syntax

```
set1.intersection(set2)
```

👉 8. `difference()`

✅ Description

Elements in first set but not in second

🔹 Syntax

```
set1.difference(set2)
```

👉 9. `issubset()`

✅ Description

Checks if set is subset

🔹 Syntax

```
set1.issubset(set2)
```

👉 10. `issuperset()`

✅ Description

Checks if set is superset

🔹 Syntax

```
set1.issuperset(set2)
```

🔹 Set Operations (Important)

```
a = {1, 2, 3}
b = {2, 3, 4}

print(a | b)   # Union
print(a & b)   # Intersection
print(a - b)   # Difference
```

📝 Interview Notes

- Set = **unique elements only**
- No indexing ❗
- Unordered collection
- `remove()` → error if not found
- `discard()` → no error
- `pop()` → random element

⚠️ Trick Questions

❓ Output?

```
s = {1, 2, 2, 3}
print(s)
```

👉 **Answer:** `{1, 2, 3}`

❓ Output?

```
s = {1, 2, 3}
s.remove(5)
```

👉 **Answer:** ❌ Error

❓ Output?

```
s = {1, 2, 3}
s.discard(5)
print(s)
```

👉 **Answer:** `{1, 2, 3}`

🎯 Common Interview Questions

1. What is a set?
2. Why sets don't allow duplicates?
3. Difference between `remove()` and `discard()` ?
4. What is union and intersection?
5. Why sets are unordered?

🧠 One-Line Revision

👉 Set = **unordered collection of unique elements**

🔁 Loops in Python (Complete Notes)

✅ Definition

A **loop** is used to **execute a block of code repeatedly** until a condition is satisfied.

🔹 Types of Loops in Python

1. **for loop**
2. **while loop**

🔁 1. `for` Loop

✅ Definition

Used to **iterate over a sequence** (list, tuple, string, range, etc.)

🔹 Syntax

```
for variable in sequence:
    # code
```

🔹 Example

```
for i in range(5):
    print(i)
```

👉 Output:

```
0 1 2 3 4
```

🔹 Loop with List

```
nums = [10, 20, 30]

for n in nums:
    print(n)
```

🔁 2. `while` Loop

✅ Definition

Executes code **while condition is True**

🔹 Syntax

```
while condition:
    # code
```

🔹 Example

```
i = 1

while i <= 5:
    print(i)
    i += 1
```

🔹 Loop Control Statements

👉 1. `break`

✅ Definition

Stops the loop completely

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

👉 Output: `0 1 2`

👉 2. `continue`

✅ Definition

Skips current iteration

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

👉 Output: `0 1 2 4`

👉 3. `pass`

✅ Definition

Does nothing (placeholder)

```
for i in range(5):
    pass
```

🔹 Nested Loop

```
for i in range(2):
    for j in range(3):
        print(i, j)
```

📝 Interview Notes

- `for` loop → used with sequences
- `while` loop → used with condition
- `break` → stop loop
- `continue` → skip iteration
- `pass` → empty block
- Avoid **infinite loops** ❗

⚠️ Trick Questions

❓ Output?

```
for i in range(3):
    print(i)
```

👉 **Answer:** `0 1 2`

❓ Output?

```
i = 0
while i < 3:
    print(i)
```

👉 **Answer:** Infinite loop ❗ (no increment)

🎯 Common Interview Questions

1. What is a loop?
2. Difference between `for` and `while`?
3. What is infinite loop?
4. Difference between `break` and `continue`?
5. What does `pass` do?

🧠 One-Line Revision

👉 Loop = **repeat code multiple times**

🔢 `range()` in Python

✅ Definition

`range()` is a **built-in function used to generate a sequence of numbers**, commonly used in loops.

🔹 Syntax

```
range(start, stop, step)
```

🔹 Parameters

- **start** → starting number (default = 0)
- **stop** → ending number (**excluded**)
- **step** → increment/decrement (default = 1)

🔹 Types of `range()`

👉 1. `range(stop)`

✅ Definition

Starts from **0** and ends at `stop - 1`

```
for i in range(5):
    print(i)
```

👉 Output:

```
0 1 2 3 4
```

👉 2. `range(start, stop)`

```
for i in range(2, 6):
    print(i)
```

👉 Output:

```
2 3 4 5
```

👉 3. `range(start, stop, step)`

```
for i in range(1, 10, 2):
    print(i)
```

👉 Output:

```
1 3 5 7 9
```

🔹 Reverse Range

```
for i in range(5, 0, -1):
    print(i)
```

👉 Output:

```
5 4 3 2 1
```

🔹 Convert to List

```
print(list(range(5)))
```

👉 Output:

```
[0, 1, 2, 3, 4]
```

📝 Interview Notes

- `range()` generates numbers (not a list directly)
- Stop value is **excluded** ❗
- Default start = 0, step = 1
- Can use **negative step for reverse**
- Mostly used with **for loop**

⚠️ Trick Questions

❓ Output?

```
print(list(range(1, 5)))
```

👉 **Answer:** `[1, 2, 3, 4]`

❓ Output?

```
print(list(range(5, 1, -1)))
```

👉 **Answer:** `[5, 4, 3, 2]`

❓ Output?

```
print(list(range(5, 5)))
```

👉 **Answer:** `[]`

🎯 Common Interview Questions

1. What is `range()` in Python?
2. Why is stop excluded?
3. Can we use negative step?
4. What does `range(5)` return?
5. Difference between `range()` and list?

🧠 One-Line Revision

👉 `range()` = **generates sequence of numbers for loops**

🔧 Functions, Recursion & Default Parameters in Python 👇

🔧 Function in Python

✅ Definition

A **function** is a **block of code that performs a specific task and can be reused**.

🔹 Syntax

```
def function_name(parameters):
    # code
    return value
```

🔹 Example

```
def greet():
    print("Hello")

greet()
```

🔹 Function with Parameters

```
def add(a, b):
    return a + b

print(add(2, 3))   # 5
```

🔁 Recursion in Python

✅ Definition

Recursion is a process where a **function calls itself**.

🔹 Important Concept

👉 Every recursive function must have:

- **Base case** (stopping condition)

- **Recursive call**

🔹 Example (Factorial)

```
def factorial(n):
    if n == 1:        # Base case
        return 1
    else:
        return n * factorial(n-1)

print(factorial(5))   # 120
```

🔹 How it Works

```
factorial(5)
= 5 * factorial(4)
= 5 * 4 * factorial(3)
...
= 5 * 4 * 3 * 2 * 1
= 120
```

⚠️ Important (Interview)

- Missing base case → ❌ infinite recursion
- Recursion uses **stack memory**
- Slower than loops in some cases

🎯 Default Parameters in Python

✅ Definition

Default parameters are values given to function parameters **at the time of function definition**, used when no argument is passed.

🔹 Syntax

```
def function_name(parameter = default_value):
```

🔹 Example

```
def greet(name="Guest"):
    print("Hello", name)

greet("John")   # Hello John
greet()         # Hello Guest
```

🔹 Multiple Default Parameters

```
def add(a=0, b=0):
    return a + b

print(add())       # 0
print(add(5))      # 5
print(add(5, 3))   # 8
```

📝 Interview Notes

- Function = **reusable block of code**
- Recursion = **function calling itself**
- Default parameters = **predefined values**
- Base case is **must in recursion**
- Functions improve **code reusability**

⚠️ Trick Questions

❓ Output?

```
def fun(a=5):
    return a

print(fun())
```

👉 **Answer:** `5`

❓ Output?

```
def fact(n):
    return n * fact(n-1)

print(fact(3))
```

👉 **Answer:** ❌ Error (no base case)

🎯 Common Interview Questions

1. What is a function?

2. What is recursion?
3. What is base case?
4. What are default parameters?
5. Difference between recursion and loop?

🧠 One-Line Revision

- Function → reusable code
- Recursion → function calls itself
- Default parameter → predefined value

📂 File I/O in Python (with Syntax & Examples) 👇

📂 File I/O in Python

✅ Definition

File I/O (Input/Output) is used to **read data from a file and write data into a file**.

🔓 Opening a File

🔹 Syntax

```
file = open("filename", "mode")
```

🔹 Modes

Mode	Description
`"r"`	Read (default)
`"w"`	Write (overwrite)
`"a"`	Append
`"x"`	Create new file
`"b"`	Binary mode

🔹 Example

```
file = open("test.txt", "r")
```

📖 Reading a File

👉 1. `read()`

✅ Reads full file

```
file = open("test.txt", "r")
data = file.read()
print(data)
file.close()
```

👉 2. `readline()`

✅ Reads one line

```
file = open("test.txt", "r")
print(file.readline())
file.close()
```

👉 3. `readlines()`

✅ Reads all lines as list

```
file = open("test.txt", "r")
print(file.readlines())
file.close()
```

✍️ Writing to a File

👉 1. Write Mode (`w`)

✅ Overwrites file

```
file = open("test.txt", "w")
file.write("Hello World")
file.close()
```

👉 2. Append Mode (`a`)

✅ Adds content at end

```
file = open("test.txt", "a")
file.write("New Line")
file.close()
```

🔒 Closing a File

✅ Definition

Releases resources used by file.

🔹 Syntax

```
file.close()
```

✅ Best Practice: `with` Statement

🔹 Syntax

```
with open("test.txt", "r") as file:
    data = file.read()
```

👉 No need to use `close()` ❗

❌ Deleting a File

🔹 Using `os` module

```
import os

os.remove("test.txt")
```

⚠️ Important Points (Interview)

- Always **close file** or use `with`
- `"w"` mode → deletes old data
- `"a"` mode → adds data
- File must exist in `"r"` mode
- Use `with` for safe handling

📝 Interview Notes

- File I/O = **read & write files**
- `open()` → open file
- `read()` → read data

- `write()` → write data
- `close()` → close file
- `with` → best practice

⚠️ Trick Questions

❓ What happens if file doesn't exist in `"r"` mode? 👉 ❌ Error

❓ Output?

```
file = open("test.txt", "w")
file.write("Hi")
file.write("Hello")
file.close()
```

👉 **Answer:** `HiHello` (no space/new line)

🎯 Common Interview Questions

1. What is File I/O?
2. Difference between `r`, `w`, `a` modes?
3. Why use `with` statement?
4. Difference between `read()` and `readline()`?
5. How to delete a file?

🧠 One-Line Revision

👉 File I/O = **reading and writing data to files**

🧠 OOP in Python (All Concepts Combined) 👇

🧠 OOP in Python

✅ Definition

Object-Oriented Programming (OOP) is a programming style that organizes code using **classes and objects**.

🔹 Class & Object

✅ Class Definition

A **class** is a **blueprint** for creating objects.

✅ Object Definition

An **object** is an **instance of a class**.

🔹 Syntax

```
class ClassName:
    pass
```

🔹 Example

```
class Student:
    name = "John"

s1 = Student()   # object
print(s1.name)
```

🔹 Class & Instance Attributes

✅ Class Attribute

Shared by all objects

✅ Instance Attribute

Unique for each object

🔹 Example

```
class Student:
    school = "ABC"   # class attribute

    def __init__(self, name):
        self.name = name   # instance attribute

s1 = Student("John")
s2 = Student("Sam")

print(s1.school)   # same
print(s2.name)     # different
```

🔹 `__init__()` Function

✅ Definition

A **constructor** that runs automatically when object is created.

🔹 Syntax

```
def __init__(self, parameters):
```

🔹 Example

```
class Car:
    def __init__(self, brand):
        self.brand = brand

c1 = Car("BMW")
```

🔹 Methods in Python

✅ Definition

Functions defined inside a class.

🔹 Example

```
class Student:
    def greet(self):
        print("Hello")

s1 = Student()
s1.greet()
```

🔹 Types of Methods

👉 1. Instance Method

- Uses `self`

```
def show(self):
    print(self.name)
```

👉 2. Static Method

✅ Definition

Method that does not use `self` or `cls`

🔹 Syntax

```
@staticmethod
def method():
```

🔹 Example

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b

print(Math.add(2, 3))
```

🔹 Important Concepts

🔐 Encapsulation

✅ Definition

Wrapping data and methods together in one unit (class)

🔹 Example

```
class Bank:
    def __init__(self, balance):
        self.balance = balance
```

🙈 Data Hiding

✅ Definition

Restricting access to data

🔹 Example

```
class Bank:
    def __init__(self):
        self.__balance = 1000   # private

    def get_balance(self):
        return self.__balance
```

🎭 Abstraction

✅ Definition

Hiding implementation details and showing only necessary parts

---

🔹 Example

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

📝 Interview Notes

- Class = blueprint
- Object = instance
- `__init__()` = constructor
- `self` = current object
- Static method → no `self`
- Encapsulation → data + methods
- Abstraction → hide complexity
- Data hiding → private variables (`__var`)

⚠️ Trick Questions

❓ What is `self` ? 👉 Reference to current object

❓ Can we access private variable directly? 👉 ❌ No (`__var`)

❓ Static method uses `self` ? 👉 ❌ No

🎯 Common Interview Questions

1. What is OOP?
2. Difference between class and object?
3. What is `__init__()` ?
4. What is encapsulation?
5. What is abstraction?
6. Difference between instance & static method?

🧠 One-Line Revision

👉 OOP = **Class + Object + Encapsulation + Abstraction**